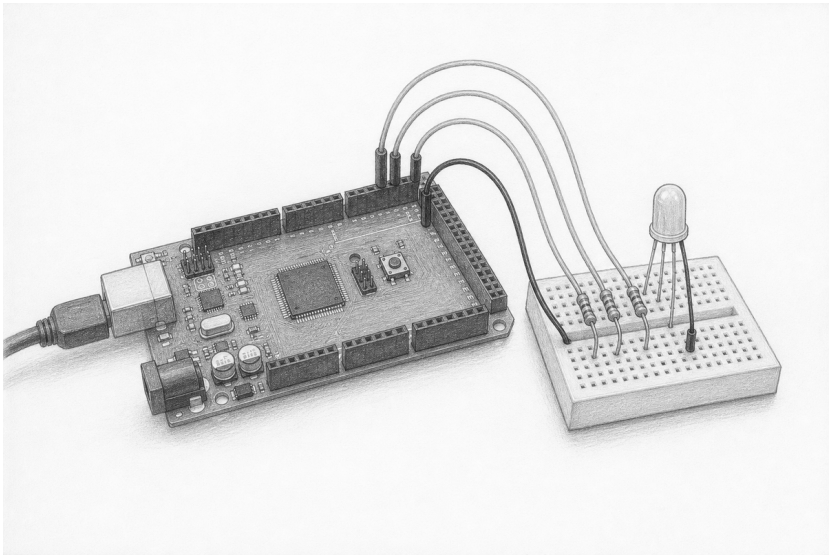
Compose three controlled currents

Lesson 004: `PwmOutput` and `RgbLed`

Calculate → wire → test → mix



Orientation plate. Use the graphite view to recognize the parts and their approximate placement. It is not a pin map. Breadboard connectivity and RGB LED leg order vary; the exact circuit and your component's datasheet govern the build.

Question	Can three owning PWM endpoints safely compose one deterministic common-cathode RGB LED?
Time	70–95 minutes
Prerequisites	Lessons 001–003; explicit status handling; resistor reading
You need	Mega 2560, USB cable, breadboard, common-cathode RGB LED, three verified resistors, four jumpers, and a multimeter
Evidence	Current calculation, exact host trace, color observations, and high-impedance teardown
Safety class	E1: USB-powered Mega and current-limited LEDs only
Status	Host-verified and experimental; not yet hardware-accepted or stable

Safety boundary

Use only USB power and the low-energy circuit shown here. Remove every power source before changing a wire. Never omit or share the three current-limiting resistors. Confirm the LED is *common cathode*; a common-anode part needs different semantics and is outside this lesson. The Mega's factory D13 LED is the independent acquisition indicator; identify its exact board path before using D13 for physical weak-bias evidence.

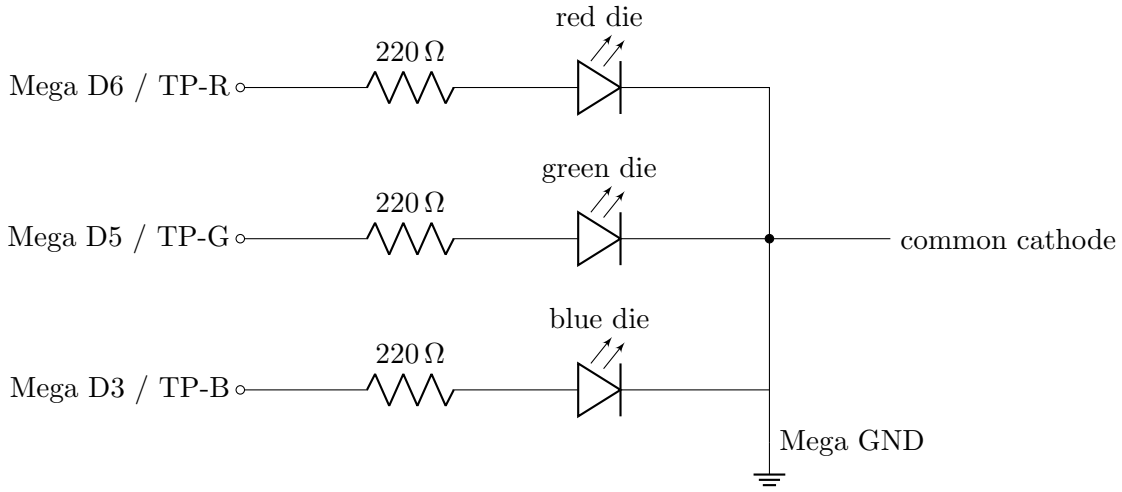
Stop for heat, odor, smoke, unexpected USB disconnects, an unknown resistor value, or an unidentified LED common lead. Disconnect USB upstream. Do not troubleshoot a powered breadboard by moving jumpers.

Check — before wiring

☐ USB removed ☐ LED datasheet or verified pinout ☐ three resistors measured ☐ rails traced

1. Read the exact common-cathode circuit

The cathodes of the three LED dies meet at one common lead connected to Mega `GND`. Each anode has its *own* series resistor. Pins 6, 5, and 3 are PWM-capable on the Mega 2560. The named measurement points are **TP-R** at D6, **TP-G** at D5, and **TP-B** at D3, each measured to Mega GND. They are on the pin side of the channel resistor. **TP-ACQ** is D13, measured to Mega GND at the board header; it belongs only to the factory acquisition LED and never carries RGB behavior.



Formal electrical schematic. The three LED symbols are the dies inside one common-cathode package. Confirm the physical leg order from that exact LED's datasheet; lens shape and lead length are not sufficient identification. TP-R, TP-G, and TP-B are measured on the Mega-pin side of their resistors. TP-ACQ remains the separate factory D13 indicator described above and is not part of this RGB network.

Check — continuity check while unpowered
☐ no pin-to-pin short ☐ each channel has one resistor ☐ common lead reaches GND only

2. Budget current before applying power

Use the LED datasheet's forward voltage V_f, not lens color or guesswork. For a first estimate,

$$I = \frac{V_{\text{pin}} - V_f}{R}.$$

With $V_{\text{pin}} = 5.0$ V and $R = 220\,\Omega$, a red die at $V_f = 2.0$ V estimates 13.6 mA; a green or blue die at 3.0 V estimates 9.1 mA. All three at full duty estimate 31.8 mA total. These are estimates, not guarantees: output voltage falls under load and device limits are simultaneous constraints.

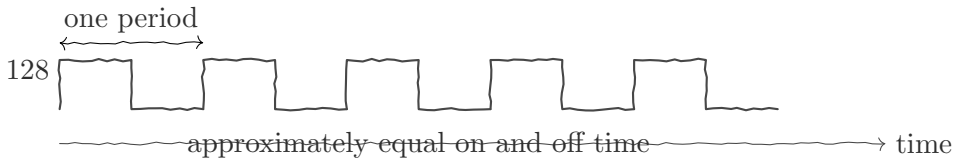
Repeat the calculation with the datasheet's *minimum* V_f, which produces the largest current. Sum all three channels at duty 255. Do not apply power unless each channel and the total remain below the documented operating limits with margin.

Channel	Measured R	Datasheet V_f	Estimated I	Accepted?
red				
green				
blue				
Total at full duty				

Do not design to the microcontroller's absolute maximum. Check the official ATmega2560 and LED datasheets for per-pin, grouped-pin, total-device, and LED limits. If any estimate lacks margin, increase resistance or stop.

3. Understand duty, not "analog voltage"

`PwmOutput::Duty` is an 8-bit value: 0 is 0% high duty, 255 is 100% high duty, and intermediate values switch rapidly. Duty 128 is about 50%, but it does not command a steady 2.5 V. In this lesson's common-cathode LED, zero is inactive. Other circuits may assign different semantics.



Predict. Order the apparent brightness for duties 0, 16, 64, 128, and 255:

---

4. Read the exact endpoint API

`PwmOutput` claims one PWM-capable Mega pin. Initialization writes the initial duty. Shutdown writes zero, selects input mode, and releases the claim. All operations are exception-free; destruction performs idempotent shutdown.

```
adk::Runtime   runtime;
adk::PwmOutput red (runtime.resources (), 6, 0);

adk::Status status = red.initialize ();
if (status.ok ())
{
    status = red.write (128);
}

red.shutdown ();
```

An invalid pin reports `InvalidPin`; a valid non-PWM pin reports `Unsupported`; a claimed pin reports `ResourceBusy`; writing before initialization reports `NotInitialized`.

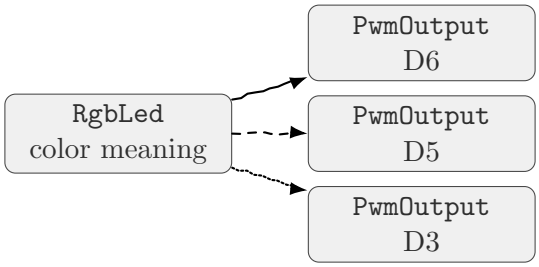
5. Compose the semantic RGB component

`RgbLed` owns three `PwmOutput` objects. Its channel descriptors retain both pin identity and the documented resistor value. For a common-cathode LED, color byte values map directly to duty.

```
adk::Runtime runtime;
adk::RgbLed  led (runtime.resources (),
                  {6, 220},
                  {5, 220},
                  {3, 220});
```

```
if (led.initialize ().ok ())
{
    led.set (adk::Rgb (255, 64, 0));
    led.off ();
}
```

Initialization is transactional: red, green, and blue claims must all succeed. A later failure releases earlier claims. `set()` updates all three channels; `off()` sets `Rgb(0,0,0)`. Shutdown proceeds through the owned endpoints, leaving every channel at zero and high impedance.



6. Prove behavior on the host

The fake Arduino trace is the electrical contract's executable evidence. Write tests for:

1. every Mega PWM pin and representative valid non-PWM pins;
2. duty endpoints 0 and 255 plus one intermediate value;
3. write before initialization and repeated initialization;
4. duplicate claims, release, and reuse;
5. destruction: `analogWrite(pin, 0)` before `pinMode(pin, INPUT)`;
6. RGB initialization failure on the second and third channel;
7. exact red–green–blue write order for several colors;
8. repeated `off()` and `shutdown()`;
9. two identical runs producing identical traces.

Trace exercise. Predict the complete operation trace for `RgbLed::initialize()`, `set(Rgb(255,64,0))`, and destruction:

7. Build and observe

Run the repository's host checks and compile the canonical Mega example before upload. Inspect every status; an ignored failure makes observations ambiguous.

The canonical sketch is finite. It claims the RGB component first and then the independent `MonoLed` on D13 transactionally: if D13 acquisition fails, all RGB claims are released. Successful acquisition produces a 250 ms TP-ACQ/D13 factory-LED pulse, then 750 ms with D13 dark. The RGB LED remains off throughout both intervals. Only then does the RGB behavior sequence show red, green, blue, white, off, and orange for 1000 ms each. It calls `shutdown()` on D13 and all RGB endpoints after the final interval and makes no further hardware calls. Acquisition, RGB behavior, and safe-state evidence are therefore distinct observations.

1. Remove USB. Build and continuity-check the exact circuit. Label the pin-side resistor nodes TP-R/D6, TP-G/D5, and TP-B/D3.
2. Connect USB only. Identify the port and upload to that exact Mega.
3. Observe TP-ACQ/D13's acquisition pulse and dark separator separately from the six RGB behavior steps. Do not count D13 as a color-identity trial, and confirm the RGB package stays off during acquisition.
4. Record each behavior color and whether any channel appears swapped. A visual match establishes only the observed package/channel mapping. Measure voltage across each channel resistor during its primary-color step and calculate current from measured resistance. During white, repeat all three drops on separate reset runs and sum the calculated currents; never move a probe while powered.
5. Wait for controlled shutdown. Host trace evidence must independently show D13 LOW followed by `INPUT`, plus duty zero followed by `INPUT` for D6, D5, and D3; darkness alone does not prove high impedance.
6. For physical safe-state evidence, remove USB and install one measured 10 kΩ weak pull-up from the measured 5 V rail to only one named TP at a time. Include the channel's 220 Ω resistor, measured LED behavior at weak current, DMM input resistance, and the board output limits in the predicted voltage ranges. Reconnect USB, record the driven-LOW voltage during the off step and the released voltage after shutdown, then remove USB before moving the fixture. Repeat independently for TP-R, TP-G, and TP-B.
7. The weak fixture current is at most $5.0\,\text{V}/10\,\text{k}\Omega = 0.50\,\text{mA}$ before instrument and LED paths. If a calculated range is ambiguous, exceeds any documented limit, or disagrees with observation, stop and leave that channel unaccepted. Repeat the same controlled method for TP-ACQ/D13 only after accounting for the exact factory LED circuit; its result is acquisition-endpoint safe-state evidence, not RGB evidence.
8. With the fixture removed and USB reconnected, press reset once. Confirm that acquisition, separation, all six behavior steps, and shutdown repeat in full.
9. Disconnect USB at the computer end. Measure the 5 V rail, TP-ACQ, TP-R, TP-G, and TP-B relative to GND and record the expected power-removed range at each. Only then touch or dismantle the circuit.

Command	Expected	Observed	Trace/status
`Rgb(255,0,0)`	red		
`Rgb(0,255,0)`	green		
`Rgb(0,0,255)`	blue		
`Rgb(255,255,255)`	combined white		
`Rgb(0,0,0)`	off		

Physical acceptance boundary. Measured bench acceptance remains a separately controlled campaign; this learner lesson does not include a bench form.

8. Explain what the abstraction prevents

1. Why must each die have a separate resistor?
2. Why does common cathode permit direct duty semantics?
3. Why should `RgbLed` compose rather than inherit from `PwmOutput`?
4. What must happen if the blue pin is already claimed?
5. Why is a stored resistor value useful even though software cannot enforce the physical resistor?
6. Which evidence proves teardown better than visual darkness?

Exit ticket

☐ circuit checked unpowered ☐ current budget recorded ☐ host tests pass ☐ Mega build passes ☐ status paths handled ☐ teardown trace verified

Companion material. Use the HTML lesson for exact source links, current API signatures, errata, Mega pin-capability tables, and authoritative datasheet links. Use this PDF for predictions, wiring checks, calculations, and observations. Physical acceptance remains in the controlled campaign.

Arduino Mega 2560 documentation Microchip ATmega2560 datasheet